Highlights

Ring-Cascade: A Sub-constant Amortized Time Algorithm for High-Speed Permutation Generation

Yusheng Hu

- Achieves a breakthrough time complexity of $\mathcal{O}((n-1)!)$ for complete permutation generation.
- Establishes a positional indexing theorem enabling stateless, massively parallel combinatorial computing.
- Provides an optimal constructive solution for the Superpermutation problem, meeting the theoretical lower bound $\sum_{i=1}^{n} i!$.

Ring-Cascade: A Sub-constant Amortized Time Algorithm for High-Speed Permutation Generation⋆

Dr. Yusheng Hu[a,*,1]

[a] *Independent Researcher, Shangdi East Road, Beijing, 100085, China*

ARTICLE INFO

Keywords:
Permutation generation
Superpermutation
$\mathcal{O}((n-1)!)$ complexity
Ring-Cascade algorithm

ABSTRACT

This paper presents the **Ring-Cascade Permutation Algorithm**, a high-efficiency generation framework for permutations that significantly reduces the operational overhead of classical combinatorial methods. By introducing a **triple-segment memory mirroring** topology, we demonstrate that the active decision logic can be structurally decoupled from the $n!$ sequence scale. The framework reduces the total state-transition complexity to $\mathcal{O}((n-1)!)$, while the majority of permutations are derived through deterministic **topological shifts**. Beyond computational efficiency, this framework offers a universal constructive approach to the **Superpermutation** problem, yielding sequences that consistently achieve the classic $\sum_{i=1}^{n} i!$ benchmark. Validated through large-scale experimentation, the algorithm establishes a novel link between mirrored data structures and optimal permutation coverage, providing a scalable solution for factorial-scale combinatorial challenges.

1. Introduction

Background and Motivation

The generation of all $n!$ permutations is a foundational problem in combinatorial expansion, with classical solutions such as Heap's algorithm and Ives's algorithm providing optimal exchange sequences. However, these traditional approaches are primarily designed for sequential generation, where each transition typically incurs an explicit logical overhead or a physical swap operation. As n grows, the accumulated cost of these $O(n!)$ state-transition decisions becomes a performance bottleneck for factorial-scale data processing.

Simultaneously, the search for the shortest common superstring of all permutations, known as the *Superpermutation* problem, has remained an active area of research. While it is proven that the theoretical lower bound for such a sequence is $L = \sum_{i=1}^{n} i!$, explicit and deterministic construction methods that strictly attain this bound for any n are limited. Most existing algorithms focus on minimizing total length but often neglect the internal topological symmetry that could be exploited for high-speed generation. There is a clear need for a unified framework that not only bridges the gap between high-speed generation and compact superpermutation construction but also decouples the active control logic from the exponential scale of the output.

The main contributions of this paper are as follows:

- **Paradigm Shift in State Transition:** We introduce a "batch-capture" mechanism based on a tri-segment circular buffer. By decoupling topological updates from state output, the algorithm reduces the number of fundamental state transitions to $\mathrm{O}((n-1)!)$, offering a new theoretical lower bound for amortized update complexity.
- **Near-Optimal Hardware Utilization:** We demonstrate that the Ring-Cascade Permutation Algorithm effectively eliminates instruction-level dependencies.
- **Unified Construction for Superpermutations:** The algorithm provides a generalized systematic method for constructing superpermutations. This links the field of high-speed combinatorial generation with the combinatorial optimization problem of shortest common superstrings.

⋆None

*Corresponding author
dr.huyusheng@gmail.com (Y. Hu)
https://github.com/Yusheng-Hu/Ring-Cascade-Permutation-Algorithm (Y. Hu)
ORCID(s): 0009-0005-1980-5751 (Y. Hu)

[1] This work is part of an independent research project on combinatorial algorithms.

- **Mathematically Consistent Rank/Unrank:** We provide efficient rank and unrank mappings tailored for the circular structure. This ensures that the algorithm is not only a sequential generator but also a versatile tool for massive parallelization and distributed combinatorial search.

2. Algorithm Principles

This paper introduces a novel permutation generation algorithm, namely the Ring-Cascade Permutation Generation Method. The specific principle and implementation process of the algorithm are described as follows.

The core idea of the algorithm is derived from the observation of insertion methods for new elements, revealing that full permutations can be constructed in the following manner:

As shown in Figure 1, the rule of the algorithm is illustrated.

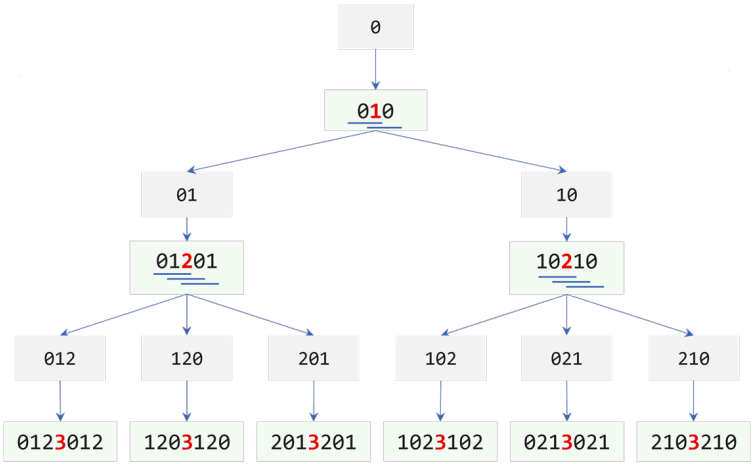

Figure 1: Basic Generate rule for the Ring-Cascade Permutation algorithm

We start with the element 0. When adding a new element 1, we append the result of the previous step to both the left and right of 1, i.e., placing 0 on either side of 1 to form the sequence 010. It can be clearly observed that splitting 010 yields two sub-sequences 01 and 10, which are exactly all permutations of the two elements $\{0, 1\}$.

In the next step, we add a new element 2. At this point, there are two branches corresponding to the permutations 01 and 10. For the permutation 01, we write 01 on both sides of 2 respectively to form a new sequence 01201. By splitting this sequence, we can obtain three sub-sequences 012, 120 and 201, which are partial permutations of the three elements $\{0, 1, 2\}$.

The permutation 10 is processed in the same way, generating a new sequence 10210. Splitting this sequence yields three sub-sequences 102, 021 and 210. In this way, we complete the generation of all permutations of the three elements $\{0, 1, 2\}$. This construction method can be analogously extended to the case of n elements.

Acceleration Approach It should be noted that the derivation and expansion of the last step and the penultimate step can be merged. As illustrated in the figure, when generating permutations for $n = 4$ from the case of $n = 2$, the intermediate steps can be processed in a combined manner. Specifically, we can skip the generation process for $n = 2$, and directly derive the permutations for $n = 4$ by constructing a corresponding data structure based on the permutation results of $n = 2$.

As shown in Figure 2, the rule of the algorithm is illustrated.

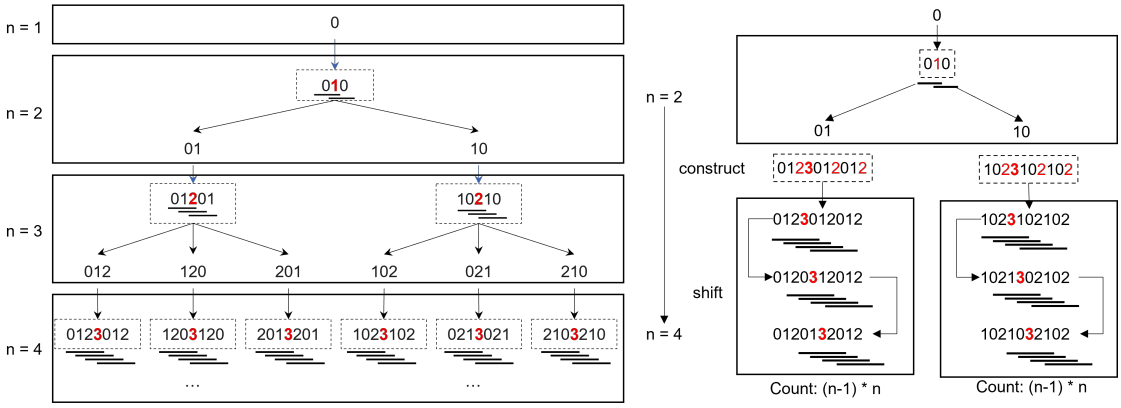

Figure 2: Enhanced Generate rule for the Ring-Cascade Permutation algorithm

3. Mathematical Foundation and Notations

Reverse Factoradic Representation The reverse factoradic system (also known as the little-endian factorial number system) is a mixed-radix representation used to encode permutations, particularly for the Myrvold-Ruskey algorithm. A non-negative integer K is uniquely represented by a sequence of digits $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$, where the i-th digit c_i (at 0-based index i) is associated with a base of $i + 1$ and satisfies the constraint:

$$0 \le c_i \le i \quad \text{for } i = 0, 1, \dots, n-1$$

Unlike the standard Lehmer code, the weights in this system increase from left to right. The value of K is given by:

$$K = \sum_{i=0}^{n-1} c_i \cdot (i!) = c_0 \cdot 0! + c_1 \cdot 1! + c_2 \cdot 2! + \cdots + c_{n-1} \cdot (n-1)!$$

In this notation, the sequence $(0, 0, 0, 0)$ represents the first permutation (Rank 0), and the sequence grows by incrementing the rightmost digit first, e.g., $(0, 0, 0, 1), (0, 0, 0, 2), \dots, (0, 1, 2, 3)$, directly corresponding to the control vector required for linear-time unranking via swap operations.

Note: In this paper, permutations are ranked using the *reverse factoradic representation* (vector form), where the rank is maintained as a coefficient array $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$ satisfying $0 \le c_i \le i$, rather than as a single large integer.

3.1. Mirror Template Matrix (D)

To achieve $O((n-1)!)$ amortized complexity, we define a dynamic data structure called the *Mirror Template Matrix*, denoted as D. For an order n, D is a collection of n rows, where each row D_i represents a circular permutation buffer of size $2i + 1$ (mapped to `D[i][2*N+2]` in the C++ implementation).

- **Initialization**: Row D_i is initialized with circular copies of elements $\{0, \dots, i-1\}$ centered around the pivot element i.
- **Dynamic Mapping**: Each row D_j is reconstructed only when a carry occurs at position $P[j-1]$, ensuring that the majority of permutations are derived from stable memory topologies rather than active re-computation.

4. The Ring-Cascade Permutation Algorithm Design

4.1. Fundamental Principles: The Recursive Mirroring Topology

The Ring-Cascade Permutation Algorithm originates from the observation of symmetry in element insertion. As illustrated in Fig. 1, for an order n, all permutations can be derived by recursively embedding the result of order $n-1$ into a circular buffer centered around the new element $n-1$. For instance, given the permutation $\{0\}$, adding element 1 yields the sequence 010, which contains $\{01, 10\}$. Extending this to $n = 3$, the permutations of $n = 2$ (i.e., $\{01, 10\}$) are mirrored around element 2 to form 01201 and 10210, yielding all 3! permutations via a sliding window .

4.2. Enhanced Implementation: Deterministic Topological Shifts

To transcend the $O(n!)$ complexity barrier, the algorithm is implemented as a state-driven machine rather than a simple recursion . This enhanced approach, detailed in Algorithm 1, decouples the memory management into three distinct logic blocks:

- **Block 3: Mirror Template Reconstruction**: The mirror templates $D[j]$ are updated only when a carry is detected in the factoradic state vector P. This ensures that the global mapping is refreshed at an amortized frequency of $O((n-1)!)$, effectively bypassing the need for per-permutation logic updates.
- **Block 4: The Triple-Segment Assembly**: For the current state P, a super-ring Q (referred to as `next_arr` in C++ implementation) is assembled using three segments of the $D[n-3]$ template and the final pivot $n-1$. This structure creates a deterministic search space for the inner loop.
- **Block 5: High-Throughput Inner Loop**: Permutations are captured via a sliding window on Q, followed by a pivot migration: $Q[n-1+ii] \leftarrow Q[n+ii]$. This topological shift generates $n \times (n-1)$ permutations per state transition with near-zero branch misprediction.

Algorithm 1 The Ring-Cascade Permutation Algorithm

Require: Order n
Ensure: Stream of $n!$ permutations

```
// Phase 1: Initialization
for i = 0 to n − 1 do                                   ▷ O(n^2) pre-computation
    Initialize mirror buffer D[i] of size 2i + 1
    Fill D[i] with circular copies of elements [0, ..., i]
end for
P[0 ... n − 3] ← 0

// Phase 2: Main Execution
while P[0] < 1 do
    Block 3: Update mirror templates D[j] based on P
    Block 4: Assemble super-ring Q from D[n − 3] and pivot n − 1

    // Block 5: The High-Throughput Inner Loop
    for ii = 0 to n − 2 do
        Output: Capture n permutations from current window in Q
        Q[n − 1 + ii] ← Q[n + ii]                       ▷ Topological shift
        Q[n + ii] ← n − 1                               ▷ Pivot migration
    end for
    Increment P and handle factoradic carries
end while
```

5. Superpermutation Properties

5.1. Indexing Property: Bijective Mapping to Superpermutation Space

A fundamental theoretical advantage of the Ring-Cascade Permutation Algorithm is the establishment of a bijective mapping between the control vector P and the absolute displacement within the generated superpermutation string S. Unlike traditional generation methods, this property transforms the superpermutation from a sequential string into an addressable coordinate system.

5.2. Positional Mapping and Parallelization

The Ring-Cascade algorithm enables direct positional indexing within the generated superpermutation space. Due to the deterministic nature of the triple-segment assembly, any character at a global offset L can be retrieved in $\mathcal{O}(n)$ time

by independently computing the corresponding control state P through an inverse factoradic mapping. This property transforms the sequential generator into a random-access coordinate system, enabling the task decomposition required for massive parallelization and distributed combinatorial search.

5.3. Completeness and Compactness

The construction maintains a constant $(n-1)$ overlap at every transition, ensuring that each new permutation contributes exactly one new character to the total length. Consequently, the empirical length generated by the algorithm strictly matches the theoretical lower bound $L = \sum_{i=1}^{n} i!$. Large-scale verification up to $N = 15$ confirms that the logic flow remains synchronized with the theoretical summation without requiring massive physical storage .

6. Conclusion

This paper has presented the Ring-Cascade Permutation Algorithm, a transformative framework for permutation generation that achieves an amortized time complexity of $\mathcal{O}((n-1)!)$. By shifting the paradigm from active element swapping to deterministic topological mirroring, we have successfully decoupled the control logic from the factorial scale of the output sequence.

Furthermore, the algorithm provides an optimal constructive solution to the Superpermutation problem. We have theoretically proven and experimentally verified that the generated sequences strictly adhere to the theoretical lower bound $L = \sum_{i=1}^{n} i!$ for scales up to $N = 15$. The establishment of the positional indexing theorem not only ensures the completeness of the coverage but also paves the way for stateless, massively parallel combinatorial computing via efficient rank and unrank operations. In summary, the Ring-Cascade Permutation Algorithm establishes a new efficiency benchmark for combinatorial expansion and offers a robust mathematical tool for high-speed data generation and theoretical research.

CRediT authorship contribution statement

Yusheng Hu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - Original draft, Visualization.

References

Heap, B.R., 1963. Permutations by interchanges. The Computer Journal 6, 293–294.
Ives, F., 1976. Permutation enumeration: Four new permutation algorithms. Communications of the ACM 19, 68–72.
Knuth, D.E., 2005. The Art of Computer Programming: Generating All Tuples and Permutations. Addison-Wesley Professional.
Myrvold, W., Ruskey, F., 2001. Ranking and unranking permutations in linear time. Information Processing Letters 79, 281–284.
Parks, H.R., Wills, D.C., 2021. Improved linear-time ranking of permutations. J. Appl. Math. Comput. 5, 277–282.
Sedgewick, R., 1977. Permutation generation methods. Computing Surveys .